

MTE 544 – Assignment 1 Report

Prepared for Prof. Yue Hu

by Will McIntyre (21009235)

NOTE: Executable code can be found at the link: https://colab.research.google.com/drive/1tRIajggihxG7fBR8A0NTx4bgl8M-0inf?usp=sharing

Question 1

Figure 1 displays the robot with a connected planar serial manipulator referenced throughout the report.

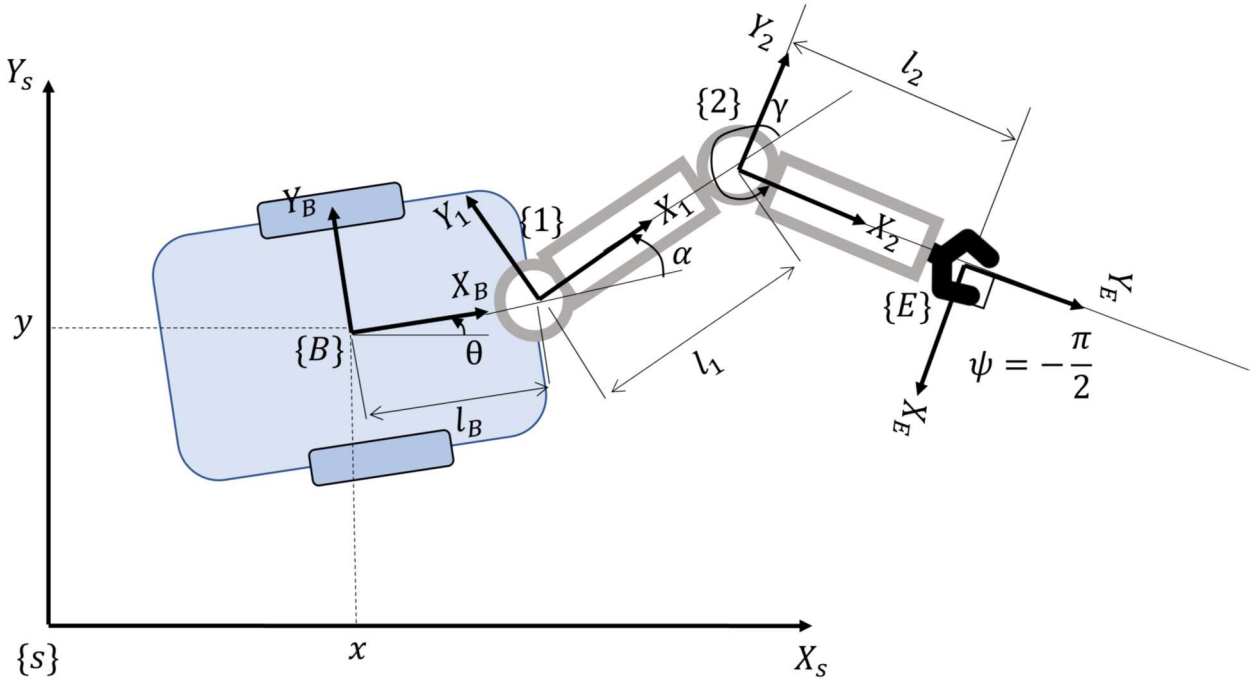


Figure 1: Mobile Robot with Planar Serial Manipulator

Question 1.a

Because there is rotation and translation between references frames, use transformation matrices with homogeneous coordinates.

To begin, determine the transformation matrix between the end effector frame and frame 2 in terms of rotation matrix R_E^2 and translation $q \in R^2$:

$$G_E^2 = \begin{bmatrix} R_E^2 & q \\ 0 & 1 \end{bmatrix}$$

Subbing in the known values:

$$G_E^2 = \begin{bmatrix} cos\psi & -sin\psi & l_2 \\ sin\psi & cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This can be done for all remaining transformation matrices in Python using the sympy library with a helper function taking in rotation angle and the translation, outputting the matrix.

$$G_2^1 = \begin{bmatrix} cos\gamma & -sin\gamma & l_1 \\ sin\gamma & cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
$$G_1^B = \begin{bmatrix} cos\alpha & -sin\alpha & l_B \\ sin\alpha & cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
$$G_B^S = \begin{bmatrix} cos\theta & -sin\theta & x \\ sin\theta & cos\theta & y \\ 0 & 0 & 1 \end{bmatrix}$$

Now with all transformation matrices known, we can compute the transformation matrix from end effector frame {E} to global frame {s} by using the composition property of homogeneous transformations:

$$G_E^S = G_B^S * G_1^B * G_2^1 * G_E^2$$

Using Python to find the simplified symbolic homogeneous transformation matrix:

$$G_E^S = \begin{bmatrix} cos(\alpha+\gamma+\psi+\theta) & -sin(\alpha+\gamma+\psi+\theta) & l_1\cos(\alpha+\theta)+l_2\cos(\alpha+\gamma+\theta)+l_B\cos(\theta)+x \\ \sin(\alpha+\gamma+\psi+\theta) & cos(\alpha+\gamma+\psi+\theta) & l_1\sin(\alpha+\theta)+l_2\sin(\alpha+\gamma+\theta)+l_B\cos(\theta)+y \\ 0 & 0 & 1 \end{bmatrix}$$

Question 1.b

To find the coordinates of the given points in the spatial frame $\{s\}$, we need to reference them in homogenous coordinates:

$$\overline{p_1}^E = \begin{bmatrix} p_1^E \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$
$$\overline{p_2}^E = \begin{bmatrix} p_2^E \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1 \\ 1 \end{bmatrix}$$

Then, we can find the homogenous coordinates of the point in the spatial frame using the previously determined transformation matrix:

$$\overline{p_1}^S = G_E^S \overline{p_1}^E$$
$$\overline{p_2}^S = G_E^S \overline{p_2}^E$$

Python was used to find the final homogenous coordinates in the spatial frame, evaluating G_E^S with the configuration details from the assignment. The homogenous form can then be converted back to cartesian coordinates:

$$\overline{p_1}^s = \begin{bmatrix} 2.2659 \\ -0.8248 \\ 1 \end{bmatrix}, p_1^s = \begin{bmatrix} 2.2659 \\ -0.8248 \end{bmatrix}$$
$$\overline{p_2}^s = \begin{bmatrix} 3.1024 \\ -1.5666 \\ 1 \end{bmatrix}, p_2^s = \begin{bmatrix} 3.1024 \\ -1.5666 \end{bmatrix}$$

Question 1.c

If there is a point in reference frame {B} that needs to be converted to a point in frame {E}, the first step is to convert it to homogenous coordinates so we can use the transformation matrices:

$$\bar{q}^B = \begin{bmatrix} q^B \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

To convert from reference frame {B} to {1}, we have to take the inverse of the transformation matrix of frame {1} to {B}:

$$G_B^1 = (G_1^B)^{-1}$$

Taking this approach for the other transformation matrices, we can get a final matrix representing the transformation from {B} to {E} using the composition and inverse properties:

$$G_B^E = G_2^E G_1^2 G_B^1 = (G_E^2)^{-1}(G_2^1)^{-1}(G_1^B)^{-1}$$

Then, to find the final coordinates:

$$\bar{q}^E = G_B^E \bar{q}^B$$

Solving using Python with the configuration described in b):

$$\bar{q}^E = \begin{bmatrix} -0.7418 \\ 0.3542 \\ 1 \end{bmatrix}, q^E = \begin{bmatrix} -0.7418 \\ 0.3542 \end{bmatrix}$$

Question 2

To create a simulation for the 2WD robot, based on inputs $[v, \omega]^T$ and obtaining poses $[x(t), y(t), \theta(t)]^T$ as outputs, begin with the state space representation of the velocity at a point P (the instantaneous center of rotation), as seen in Figure 2.

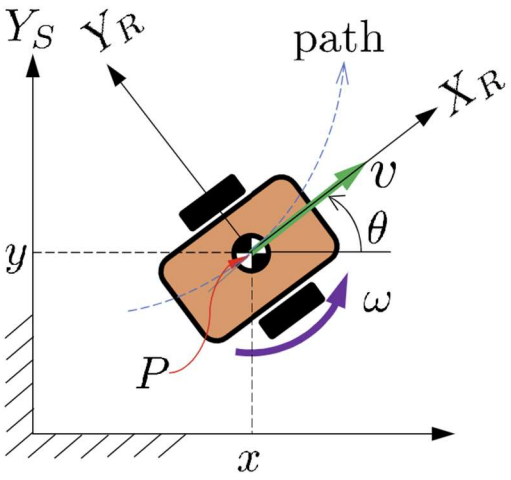


Figure 2: Motion Model of 2WD Robot

$$\dot{x} = v\cos\theta$$
$$\dot{y} = v\sin\theta$$
$$\dot{\theta} = \omega$$

Question 2.a

To simulate and plot the trajectories for different velocity profiles, we can use Python's scippy library to integrate an initial value problem and solve this system of ODE's using explicit Runge-Kutta method. Using 0.1 second intervals and assuming initial conditions of $x = 0\ m, y = 0\ m, \theta = 0\ rad$, we can plot the trajectory and each variable against time.

Question 2.a.i.1

Figure 3 below shows the 2D trajectory and pose of the robot over time when linear velocity is 1 m/s and angular velocity is 0 rad/s. As expected, the plots show the robot translating in the x direction over time and ending at x = 30 m. Because there is no rotational velocity, the robot does not change directions and moves straight.

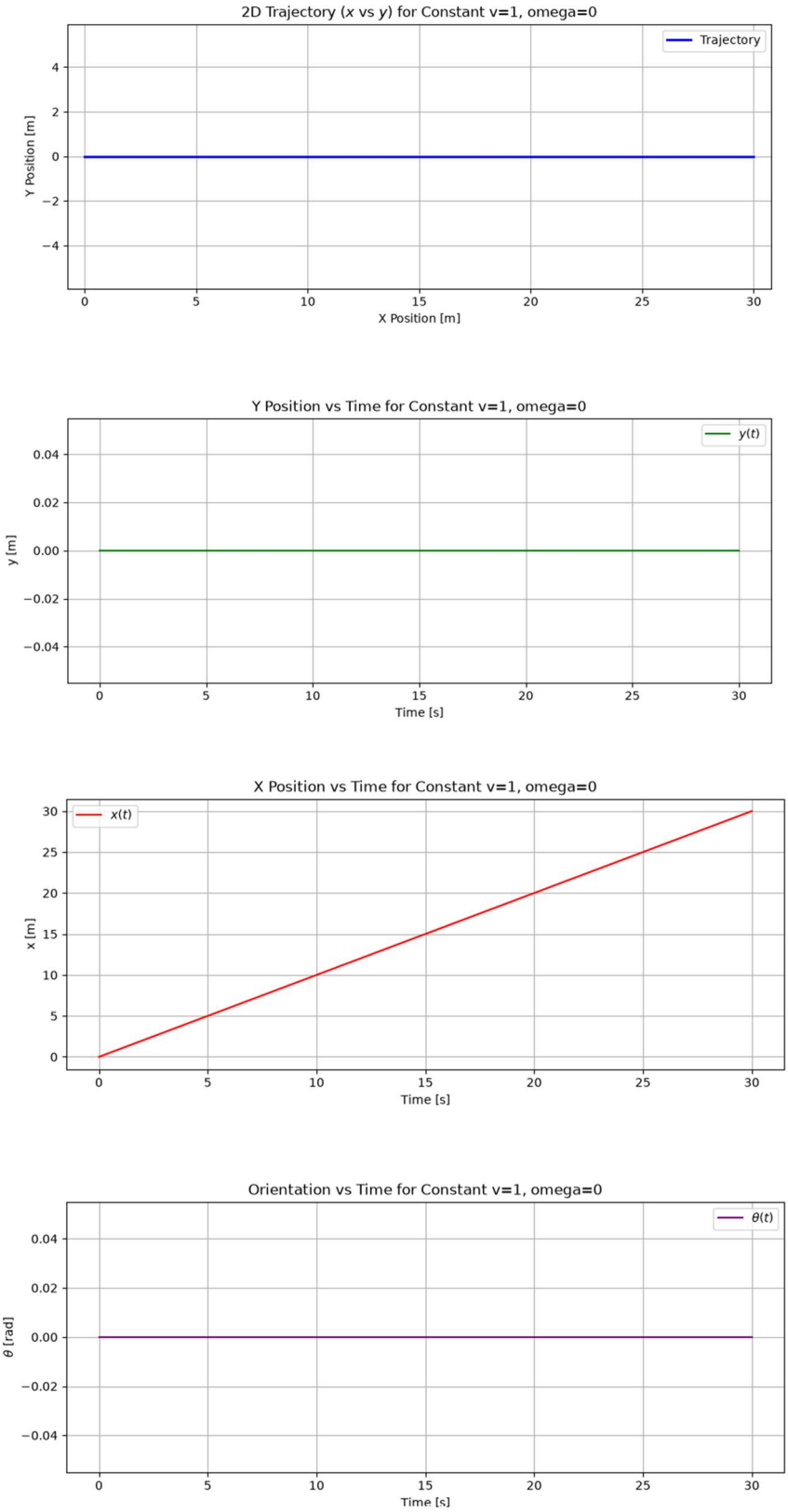


Figure 3: 2D Trajectory and Pose of Robot with Constant v = 1 m/s and ω = 0 rad/s

Question 2.a.i.2

Figure 4 shows the 2D trajectory and pose of the robot over time when linear velocity is 0 m/s and angular velocity is 1 rad/s. Once again, the plots show the expected behaviour. X and Y position remain at zero as there is no translation, purely rotation. The orientation maxes out at 15 radians after the full 30 seconds and increases linearly.

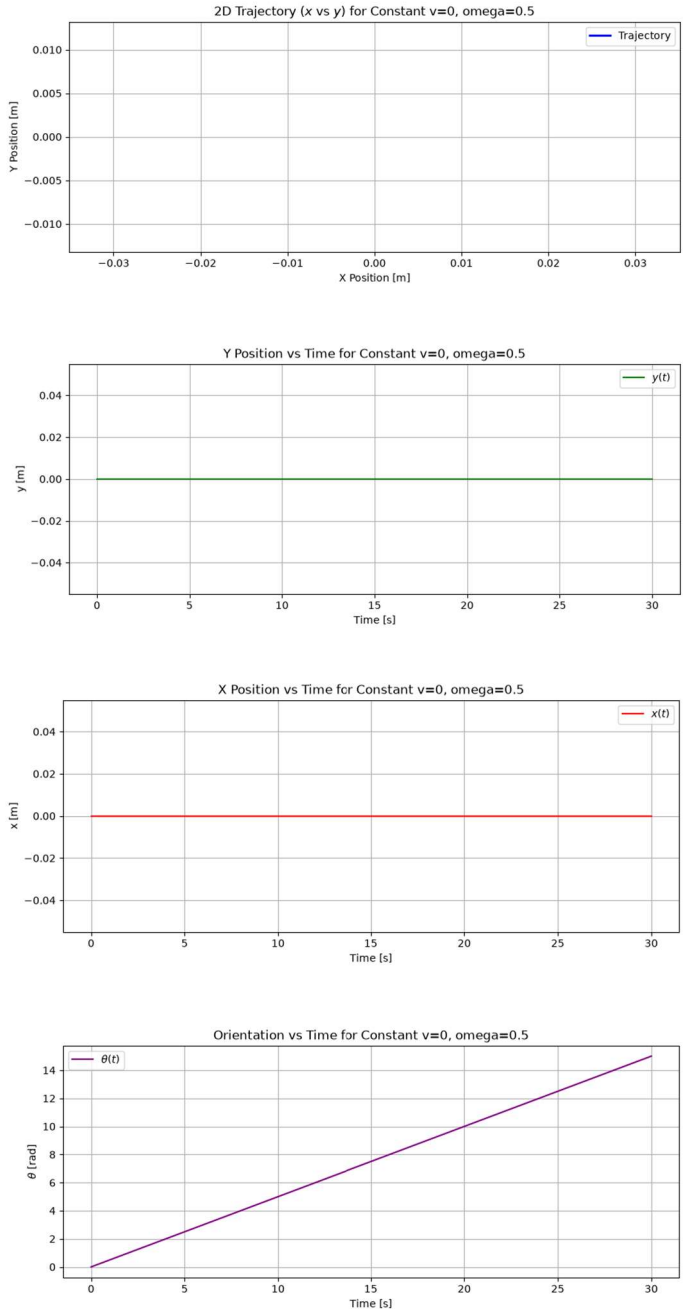


Figure 4: 2D Trajectory and Pose of Robot with Constant v = 0 m/s and ω = 0.5 rad/s

Question 2.a.i.3

Next, Figure 5 displays the 2D trajectory and pose over time when the linear velocity is 1 m/s and angular velocity is 0.5 rad/s. Looking at the plots, we can see that the robot's trajectory is a circle of radius 2 meters, the X and Y position fluctuate with an amplitude of 2m and the orientation is linear. This is intuitive as there is both linear and angular velocity that causes the robot to both translate and rotate, and when these are constant it should have a clear and repeated shape.

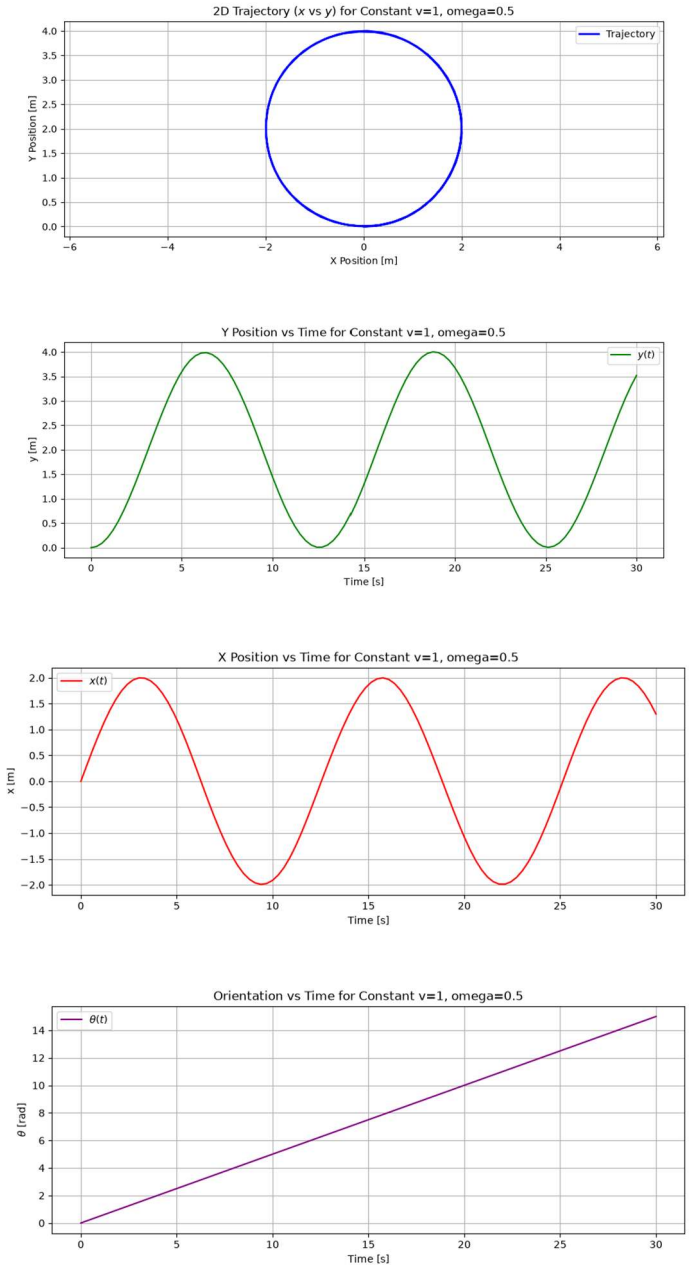


Figure 5: 2D Trajectory and Pose of Robot with Constant v = 1 m/s and ω = 0.5 rad/s

Question 2.a.ii

Finally, when the velocity profile is not constant, and instead depends on time as described in the assignment, we get different behaviour shown in Figure 6 below. Because of the varying speed and turning rate, the robot does not travel in a simple geometric shape. The X and Y graphs show smooth, wave-like progressions rather than sine waves or linear ramps, reflecting the forward and weaving movement on the 2D plane. The orientation plot in this example shows a wavy upward trend. This indicates that the robot speeds up and slows down its rate of turning as time progresses

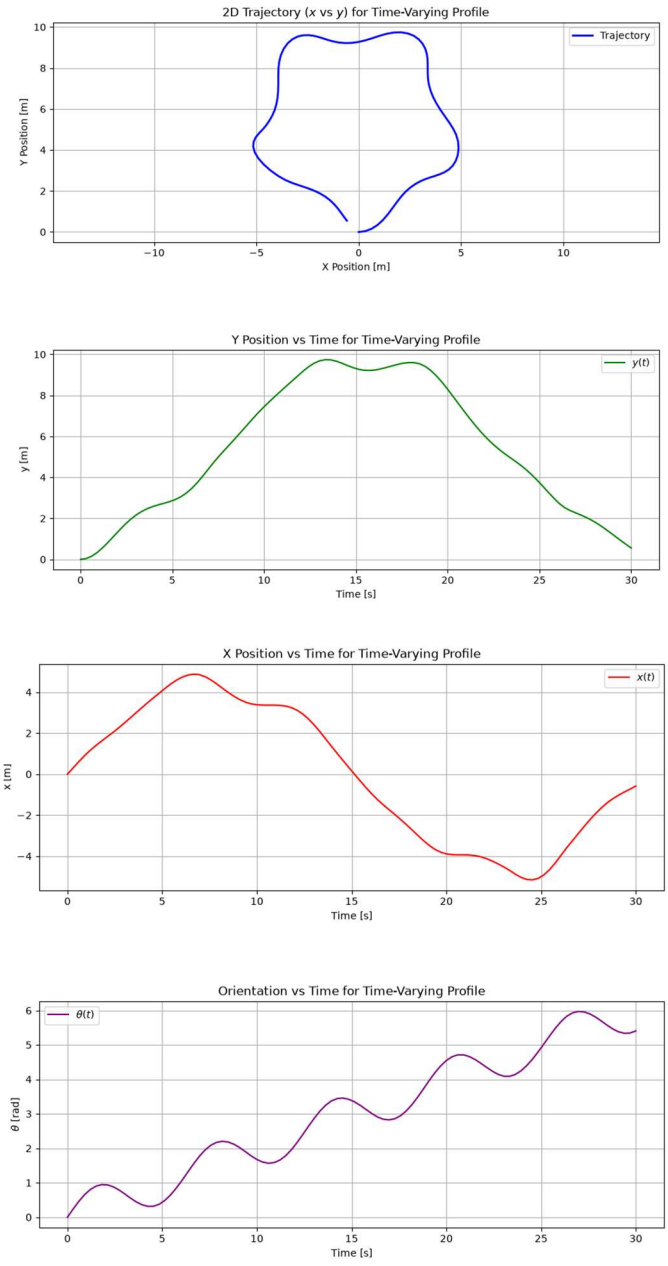


Figure 6: 2D Trajectory and Pose of Robot with Time-Varying Velocity Profile

Question 2.b

For a standard differential-drive mobile robot, the relationship between linear velocity v, angular velocity ω, track width T, wheel radius r and the right/left wheel rotational speeds (u_r, u_l) is given by:

$$v = \frac{r}{2}(u_r + u_l) \quad (1)$$

$$\omega = \frac{r}{T}(u_r - u_l) \quad (2)$$

To find the wheel speeds given v and ω, rearrange the above equations:

$$u_r = \frac{v + \frac{T}{2}\omega}{r}$$

$$u_l = \frac{v - \frac{T}{2}\omega}{r}$$

With T = 0.3m and r = 0.15m, we can find wheel speeds needed to achieve the constant velocities in 2.a.i shown in Table 1 below:

Table 1: Constant Velocities and Associated Wheel Speeds

Velocities $[v, \omega]^T$	**u_l [rad/s]**	**u_r [rad/s]**
$[1, 0]^T$	6.67	6.67
$[0, 0.5]^T$	-0.5	0.5
$[1, 0.5]^T$	6.17	7.17

To adopt the simulation for wheel speed inputs, a transformation must be added at the beginning. Instead of directly reading the linear and angular velocities, the ODE function would take $u_l(t)$ and $u_r(t)$ and compute the linear and angular velocities as a function of time using the relations described in Equations (1) and (2). The rest of the numerical integration process remains the same.

Question 3

Figure 7 displays the reference frames and position/velocity for a single omnidirectional wheel.

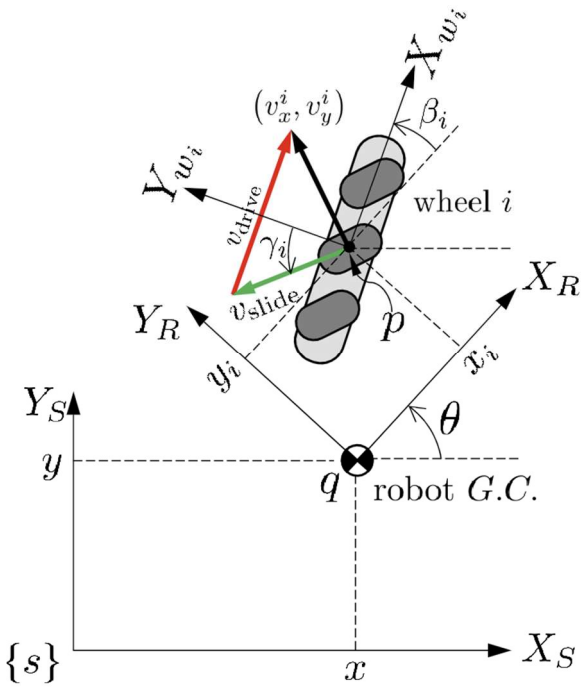


Figure 7: Reference Frames and Kinematic Variables for Single Wheel

Figure 9 displays the three wheeled configuration and the associated reference planes.

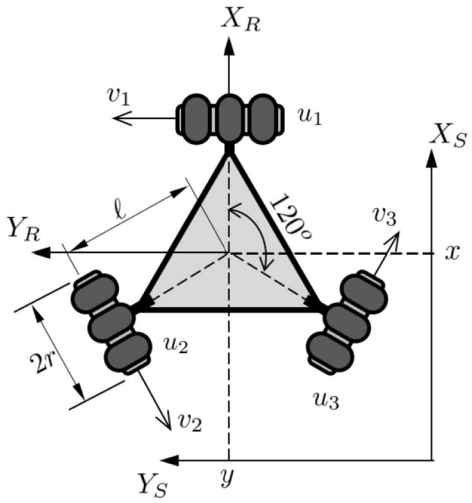


Figure 8: 3-Wheeled Omnidirectional Mobile Robot

Question 3.a

To derive the equation of motion for the three-wheeled omnidirectional robot, we want to relate the robot frame's $\{\dot{x}, \dot{y}, \dot{\theta}\}$ with the wheel speed $\{v^i\}^{w_i}$ where w_i is the reference frame of the wheel.

To begin, using the coordinate TF for velocity where $q = [x\ y]^T$ is the robot's geometric center's position and p_i^s and p_i^r are the points of the i'th wheel's reference frame in $\{s\}$ and $\{r\}$, respectably:

$$(v_i)^s = \hat{w}(p_i^s - q) + \ v = \hat{w}R_r^s p_i^r + v \tag{3}$$

From the relation between $(v_i)^{w_i} and\ (v_i)^s$:

$$(v_i)^s = R_{w_i}^s (v_i)^{w_i} = R_r^s R_{w_i}^r (v_i)^{w_i} \tag{4}$$

The rotation matrices between the robot/global frame (with angle θ) and wheel/robot frames (angle β_i) is:

$$R_r^s = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix}$$
$$R_{w_i}^r = \begin{bmatrix} cos\beta_i & -sin\beta_i \\ sin\beta_i & cos\beta_i \end{bmatrix}$$

Combining Equation (3) and (4) yields:

$$(v_i)^{w_i} = \left(R_{w_i}^r\right)^{-1} (R_r^s)^{-1} (\hat{w}R_r^s p_i^r + v)$$

We can then map robot body velocities to wheel frame in a general form using $\dot{q} = \left[\dot{x}\ \dot{y}\ \dot{\theta}\right]^T$:

$$(v_i)^{w_i} = g_i(\theta)\dot{q}$$

Where $g_i(\theta)$ includes the point $p_i^r = [x_i\ y_i]^T$:

$$g_i(\theta) = \begin{bmatrix} \cos(\theta + \beta_i) & \sin(\theta + \beta_i) & x_i \sin\beta_i - y_i \cos\beta_i \\ -\sin(\theta + \beta_i) & \cos(\theta + \beta_i) & x_i \cos\beta_i + y_i \sin\beta_i \end{bmatrix}$$

Then, we can map the motor speed to the wheel speeds using the assumption that all wheel radii are the same, and the fact that for 3 wheeled omnidirectional robots, $\gamma_i = 0°$ for all wheels:

$$u_i = \frac{1}{r} v_{drive} = \frac{1}{r}\left((v_x)_i^{w_i} + \left(v_y\right)_i^{w_i} tan\gamma_i\right)$$
$$= \frac{1}{r}\left((v_x)_i^{w_i} + \left(v_y\right)_i^{w_i} \tan(0°)\right)$$

$$u_i = \frac{(v_x)_i^{w_i}}{r}$$

Knowing the equation for individual wheel speed and motor speed, we can combine the wheel equations for all 3 wheels:

$$u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = G(\theta)\dot{q}$$

$G(\theta)$ is constructed by stacking each wheel's $g_i(\theta)$ matrix's first row multiplied by $\frac{1}{r}$:

$$u = \frac{1}{r}\begin{bmatrix} \cos(\theta+\beta_1) & \sin(\theta+\beta_1) & x_1\sin\beta_1 - y_1\cos\beta_1 \\ \cos(\theta+\beta_2) & \sin(\theta+\beta_2) & x_2\sin\beta_2 - y_2\cos\beta_2 \\ \cos(\theta+\beta_3) & \sin(\theta+\beta_3) & x_3\sin\beta_3 - y_3\cos\beta_3 \end{bmatrix}\dot{q}$$

Note that we know the B_i angles between the wheel and robot reference frames, and they are constant.

$$\beta_1 = 90° = \frac{\pi}{2}\ rad$$
$$\beta_2 = 90° + 120° = \frac{7\pi}{6} rad$$
$$\beta_3 = 90° - 120 = -\frac{\pi}{6} rad$$

Note that we also know the p_i^r points that represent the locations of the wheels to the geometric center of the robot from distance l:

$$p_1^r = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} l \\ 0 \end{bmatrix}$$
$$p_2^r = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} lcos(\frac{2\pi}{3}) \\ lsin(\frac{2\pi}{3}) \end{bmatrix}$$
$$p_3^r = \begin{bmatrix} x_3 \\ y_3 \end{bmatrix} = \begin{bmatrix} lcos(-\frac{2\pi}{3}) \\ lsin(-\frac{2\pi}{3}) \end{bmatrix}$$

Subbing in these known angles and points, and using Python to simplify the matrix, we get the final form of the equation of motion.

$$u = \frac{1}{r}\begin{bmatrix} -\sin(\theta) & \cos(\theta) & l \\ -\cos\left(\theta+\frac{\pi}{6}\right) & -\sin\left(\theta+\frac{\pi}{6}\right) & l \\ \sin\left(\theta+\frac{\pi}{3}\right) & -\cos\left(\theta+\frac{\pi}{3}\right) & l \end{bmatrix}\dot{q} \qquad (5)$$

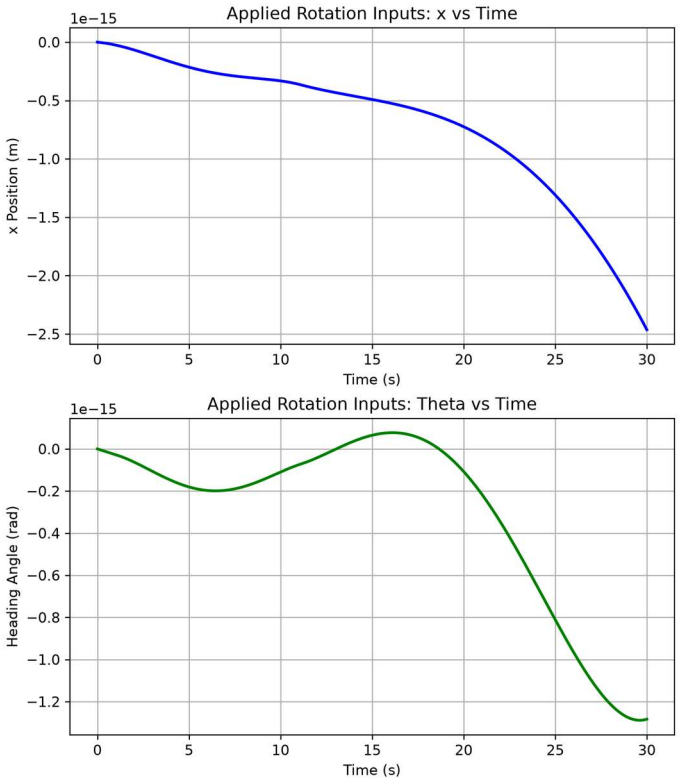
Question 3.b

To find an equation for $\dot{q}$, we can invert Equation (5):

$$\dot{q} = r \begin{bmatrix} -\sin(\theta) & \cos(\theta) & l \\ -\cos\left(\theta + \frac{\pi}{6}\right) & -\sin\left(\theta + \frac{\pi}{6}\right) & l \\ \sin\left(\theta + \frac{\pi}{3}\right) & -\cos\left(\theta + \frac{\pi}{3}\right) & l \end{bmatrix}^{-1} u$$

Question 3.b.I

Applying the known values of $r = 10\ cm$ and $l = 25\ cm$, along with $u_1 = -2\ rad/s$, $u_2 = 1\ rad/s$, $u_3 = 1\ rad/s$ and using scipy's solve_ivp function with initial conditions $x = 0\ m$, $y = 0\ m$, $\theta = 0\ rad$ leads to plots showing the pose and 2D motion of the robot shown in Figure 9. We can see that the robot is moving straight in the y direction and the plots display this. Note the graph plotting x location over time is at a very small scale (due to floating point errors in the ODE solving library).



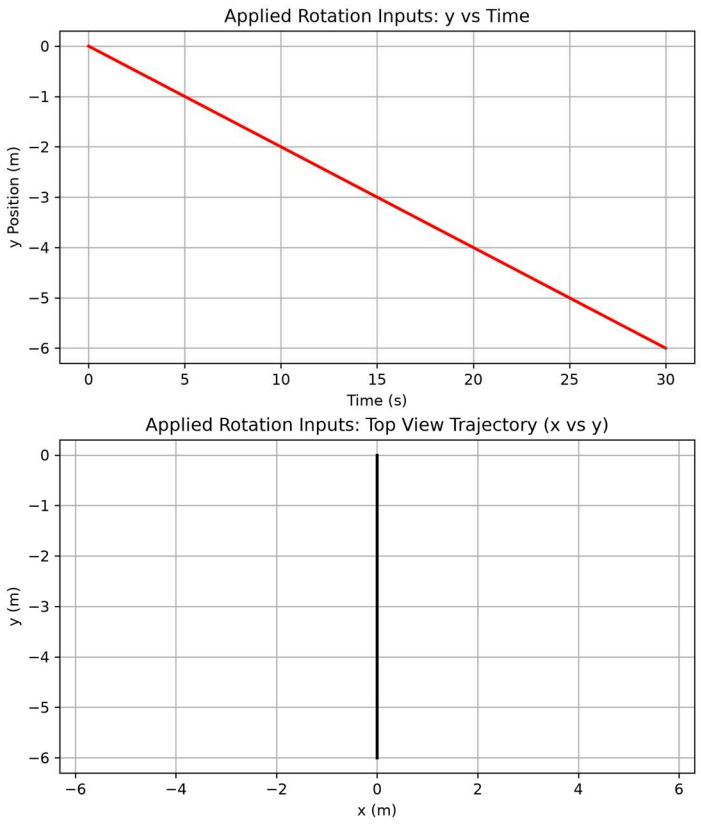


Figure 9: Pose and 2D top view Trajectory from Constant Rotation Inputs

Question 3.b.II

To design constant control signals that allows the robot to move in the desired paths, use Equation (5) and evaluate it with the given velocities $\dot{q}$ at $t = 0$, setting $\theta = 0$.

To move in a straight line from the origin with a slope of 60 degrees, the velocity vector follows:

$$\dot{q} = \begin{bmatrix} v_{straight} sin(\frac{\pi}{3}) \\ v_{straigh}\ cos(\frac{\pi}{3}) \\ 0 \end{bmatrix}$$

Setting $v_{straight}$ to an arbitrary value of 0.2 m/s and then solving Equation (5) at the initial condition $\theta = 0$ in Python led to the following wheel speeds:

$$u_1 = 1.732 \frac{rad}{s}$$
$$u_2 = -1.732 \frac{rad}{s}$$
$$u_3 = 0 \frac{rad}{s}$$

Based on these wheel speeds, Figure 10 displays the pose and 2D trajectory for the robot, confirming that the behaviour is expected. Note the very small scale in the θ graph which is because of floating point precision in the solve_ivp ODE solver.

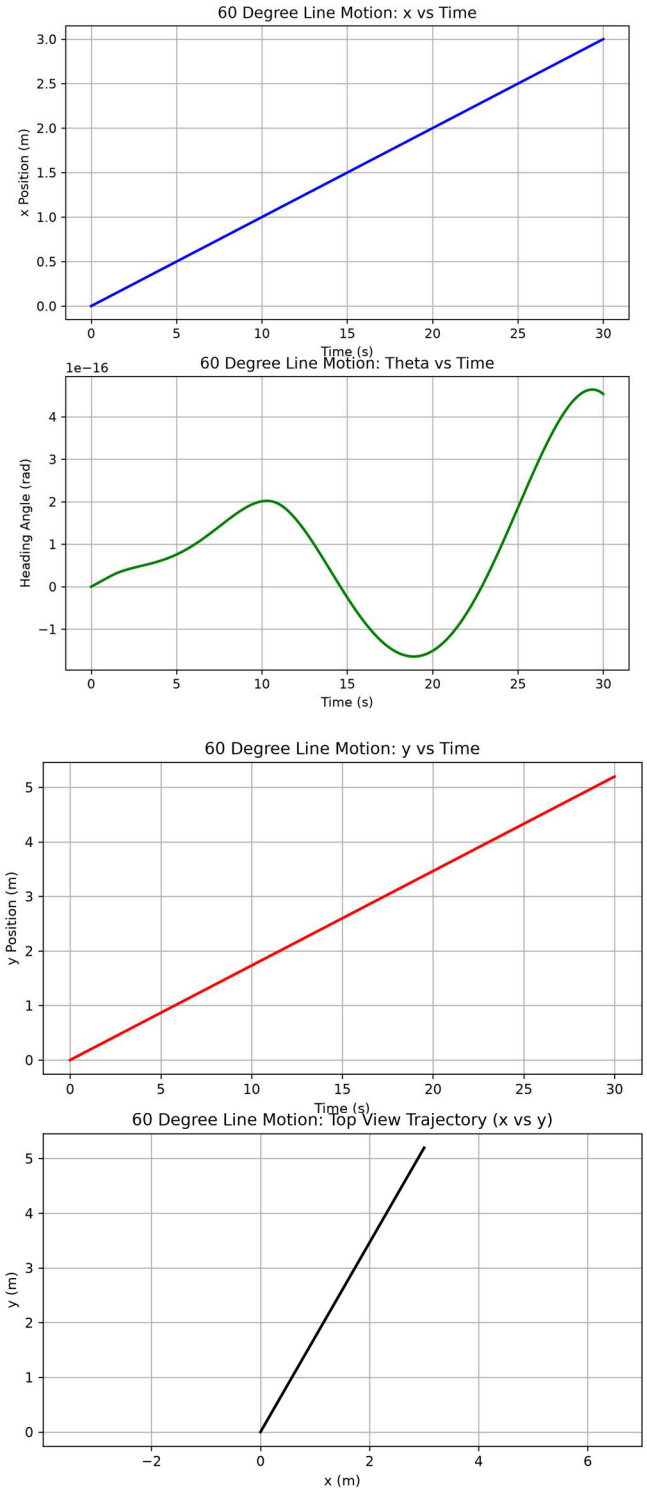


Figure 10: Pose and 2D top view Trajectory from 60 Degree Linear Motion

To move in a circle of diameter 2m ($r = 1m$), we can set an arbitrary linear velocity requirement of 0.3 m/s and then relate ω to v:

$$\omega = \frac{v}{r} = \frac{v}{1\,m} = 0.3\frac{rad}{s}$$

Then, to evaluate Equation (5) at the initial condition to determine constant wheel speeds, the instantaneous velocity vector $\dot{q}$ is required. At the initial condition, the starting point must have the pure forward translation (assigned to the x-axis in the local frame seen in Figure 9) and the rotational spin.

$$\dot{q} = \begin{bmatrix} 0.3 \\ 0 \\ 0.3 \end{bmatrix}$$

Solving for the wheel speeds using Equation (5) at the initial condition $\theta = 0$ led to the following wheel speeds:

$$u_1 = 0.5\ \frac{rad}{s}$$
$$u_2 = -1.232\ \frac{rad}{s}$$
$$u_3 = 2.232\ \frac{rad}{s}$$

Figure 11 displays the pose and 2D trajectory of the robot, found using the solve_ivp function from scipy again. These graphs once again confirm the expected behaviour as the 2D trajectory displays a circular shape and oscillation x and y locations over time, along with an increasing orientation angle.

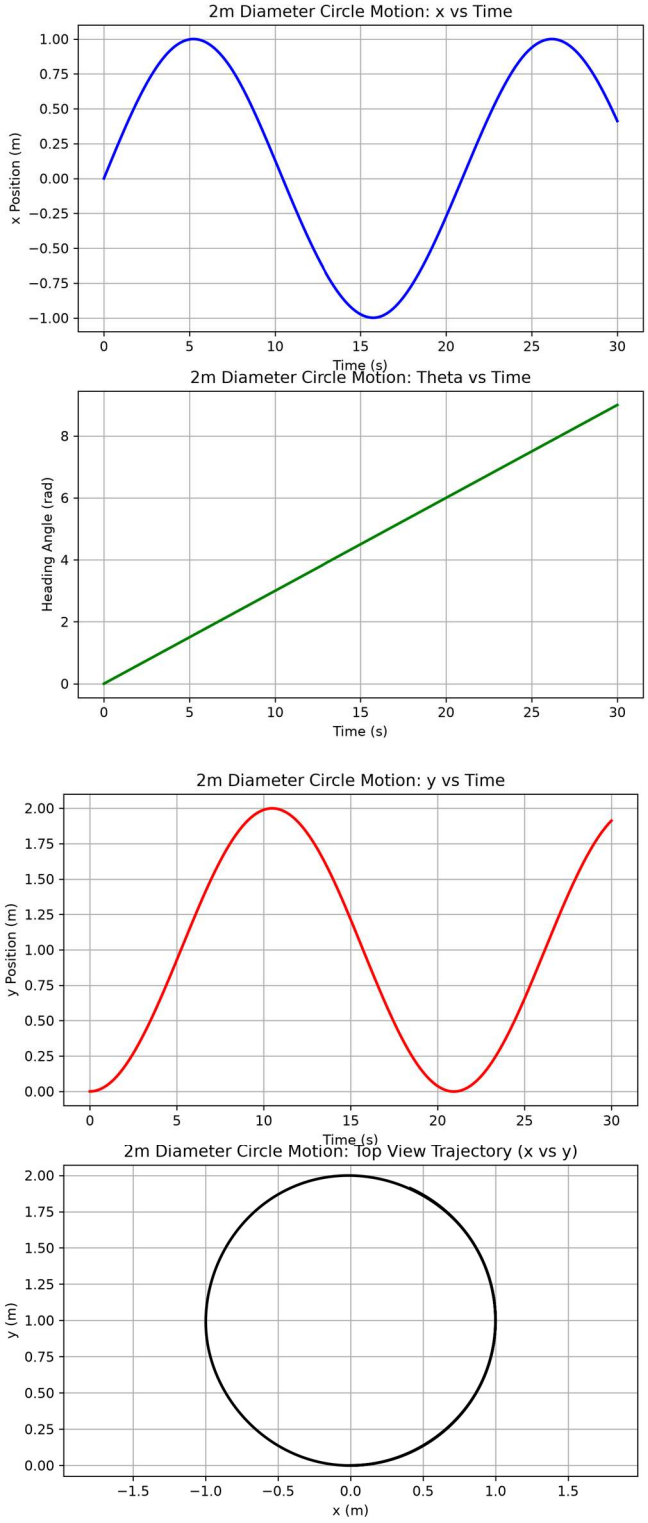


Figure 11: Pose and 2D top view Trajectory from 2m Diameter Circular Motion